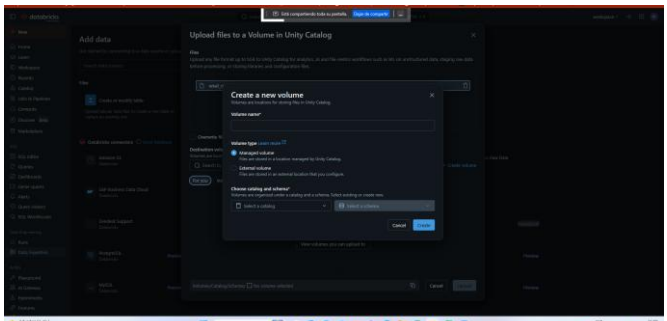
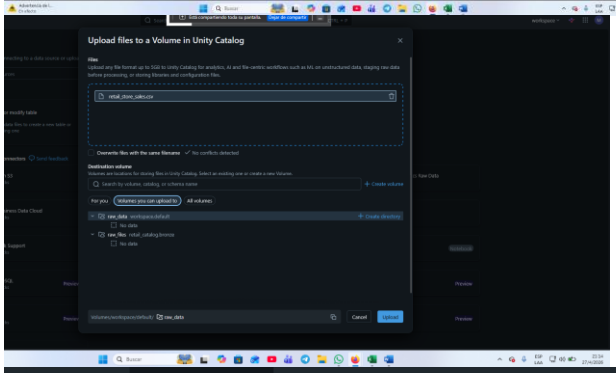
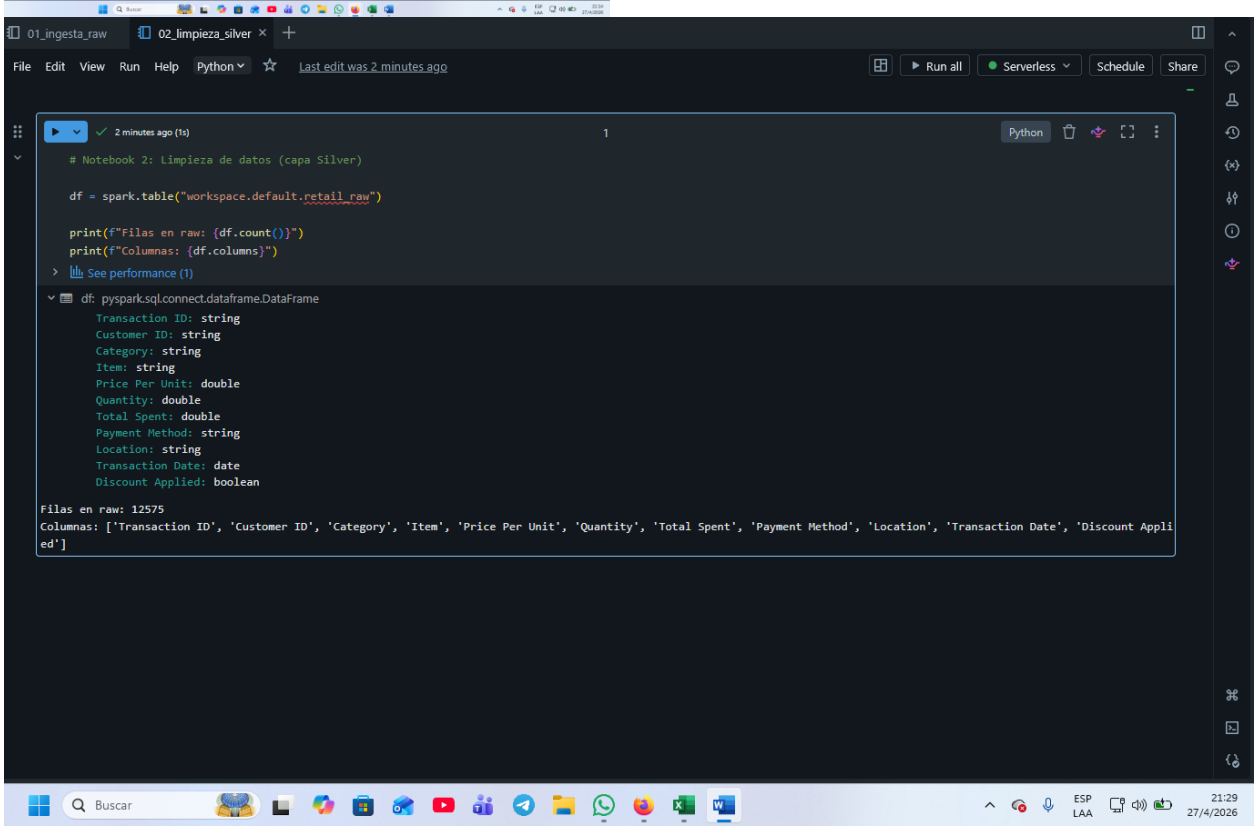
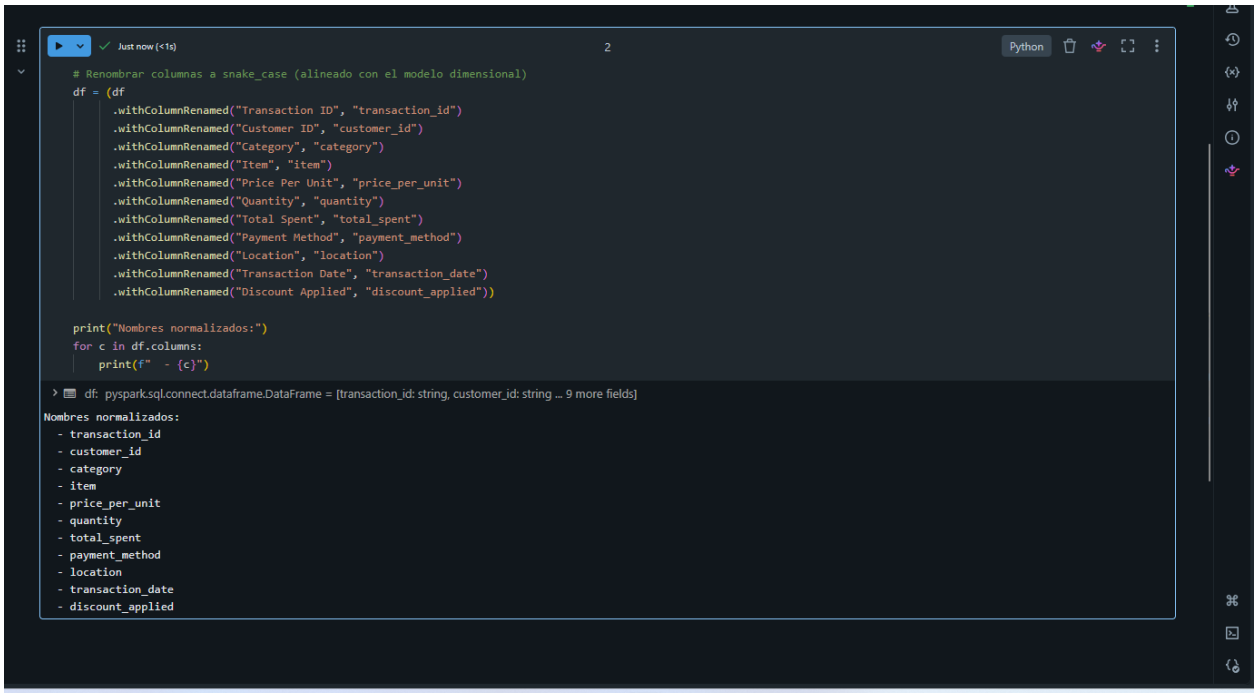
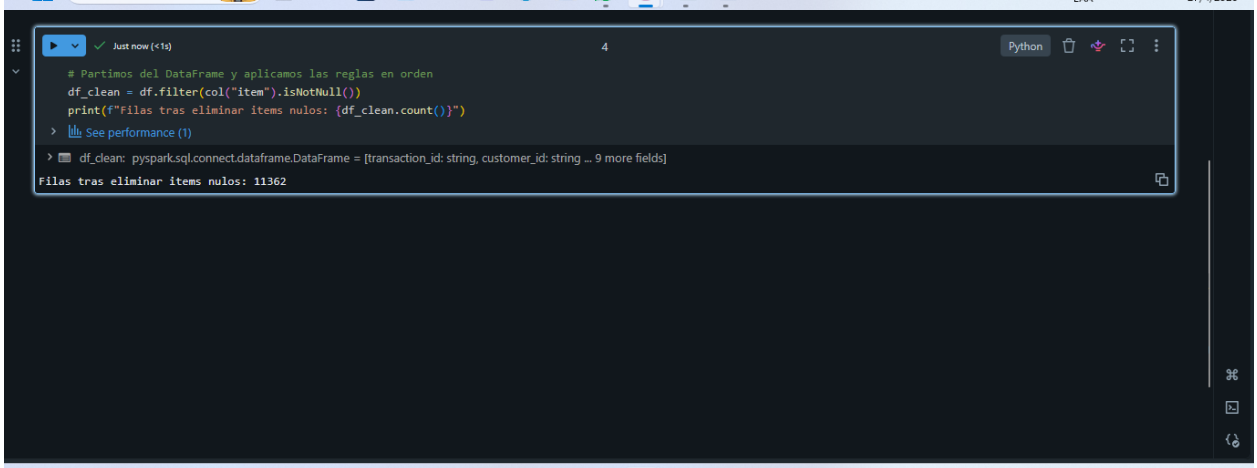
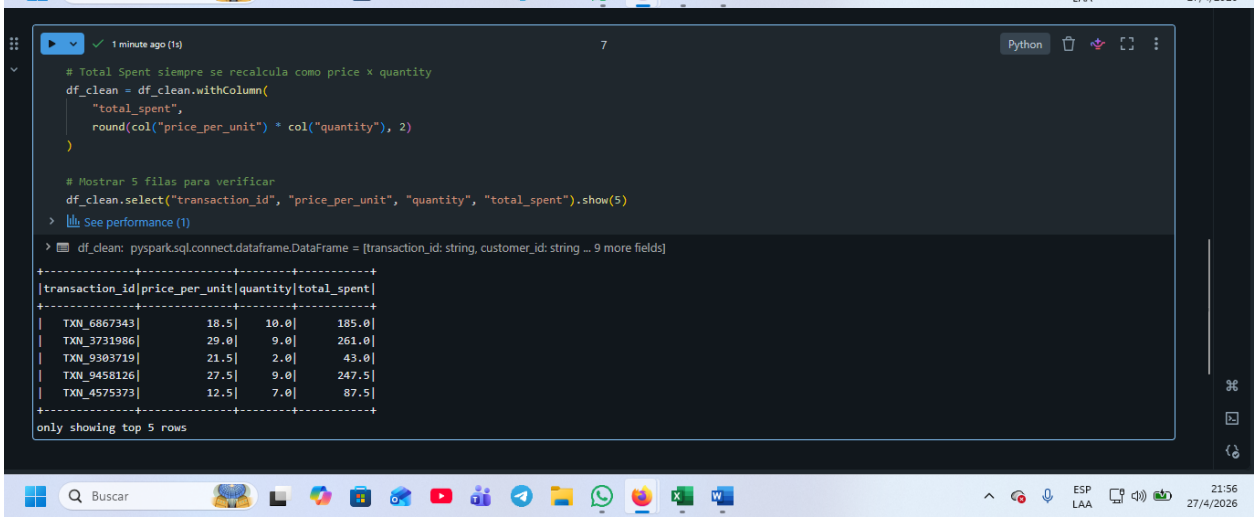

Upload files to a Volume in Unity Catalog
Create a new volume

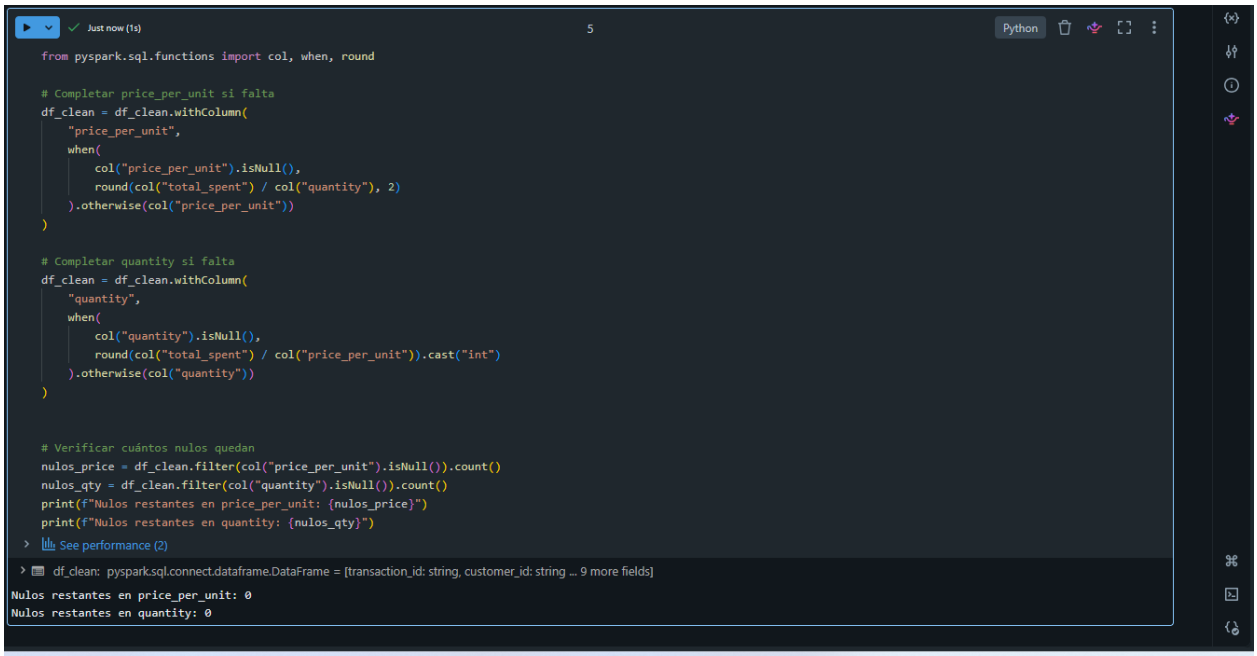

Upload files to a Volume in Unity Catalog

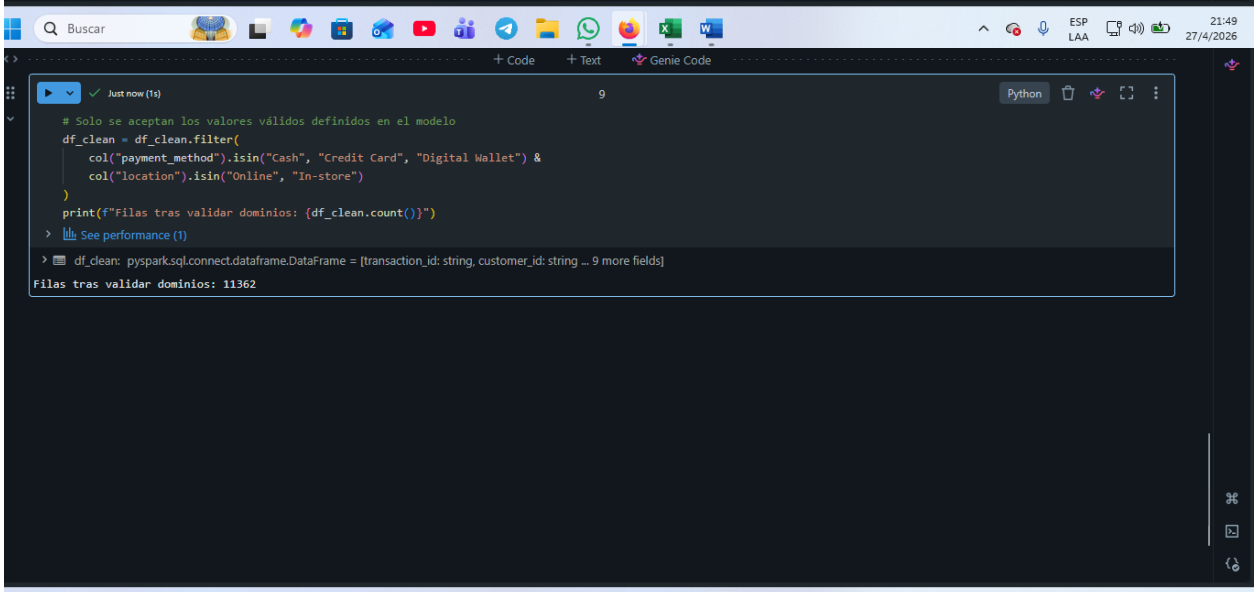

01_ingesta_raw
02_limpieza_silver
File Edit View Run Help Python
Last edit was 2 minutes ago
Run all
Serverless
Schedule
Share
2 minutes ago (1s)
1
Python
# Notebook 2: Limpieza de datos (capa Silver)
df = spark.table("workspace.default.retail_raw")
print(f"Filas en raw: {df.count()}")
print(f"Columnas: {df.columns}")
See performance (1)
df: pyspark.sql.connect.dataframe.DataFrame
Transaction ID: string
Customer ID: string
Category: string
Item: string
Price Per Unit: double
Quantity: double
Total Spent: double
Payment Method: string
Location: string
Transaction Date: date
Discount Applied: boolean
Filas en raw: 12575
Columnas: ['Transaction ID', 'Customer ID', 'Category', 'Item', 'Price Per Unit', 'Quantity', 'Total Spent', 'Payment Method', 'Location', 'Transaction Date', 'Discount Applied']
Buscar
ESP
LAA
21:29
27/4/2026

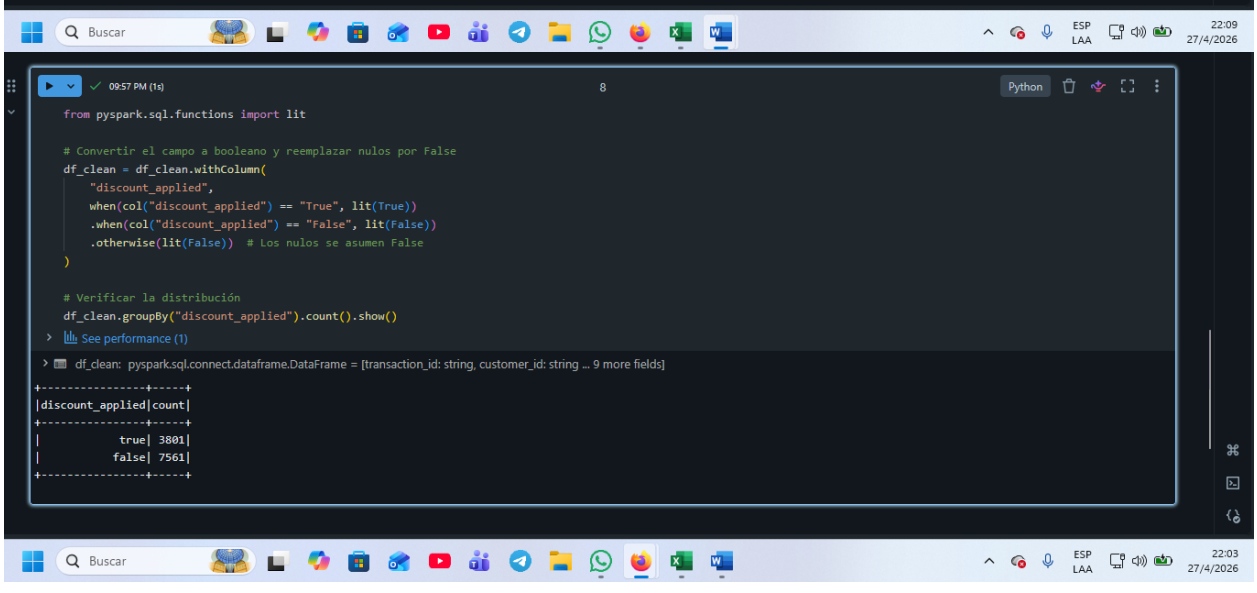

Just now (<1s)
2
Python
# Renombrar columnas a snake_case (alineado con el modelo dimensional)
df = (df
.withColumnRenamed("Transaction ID", "transaction_id")
.withColumnRenamed("Customer ID", "customer_id")
.withColumnRenamed("Category", "category")
.withColumnRenamed("Item", "item")
.withColumnRenamed("Price Per Unit", "price_per_unit")
.withColumnRenamed("Quantity", "quantity")
.withColumnRenamed("Total Spent", "total_spent")
.withColumnRenamed("Payment Method", "payment_method")
.withColumnRenamed("Location", "location")
.withColumnRenamed("Transaction Date", "transaction_date")
.withColumnRenamed("Discount Applied", "discount_applied"))
print("Nombres normalizados:")
for c in df.columns:
print(f" - {c}")
df: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
Nombres normalizados:
- transaction_id
- customer_id
- category
- item
- price_per_unit
- quantity
- total_spent
- payment_method
- location
- transaction_date
- discount_applied
Buscar
ESP
LAA
21:32
27/4/2026


Just now (<1s)
4
Python
# Partimos del DataFrame y aplicamos las reglas en orden
df_clean = df.filter(col("item").isNotNull())
print(f"Filas tras eliminar items nulos: {df_clean.count()}")
See performance (1)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
Filas tras eliminar items nulos: 11362
Buscar
ESP
LAA
21:45
27/4/2026

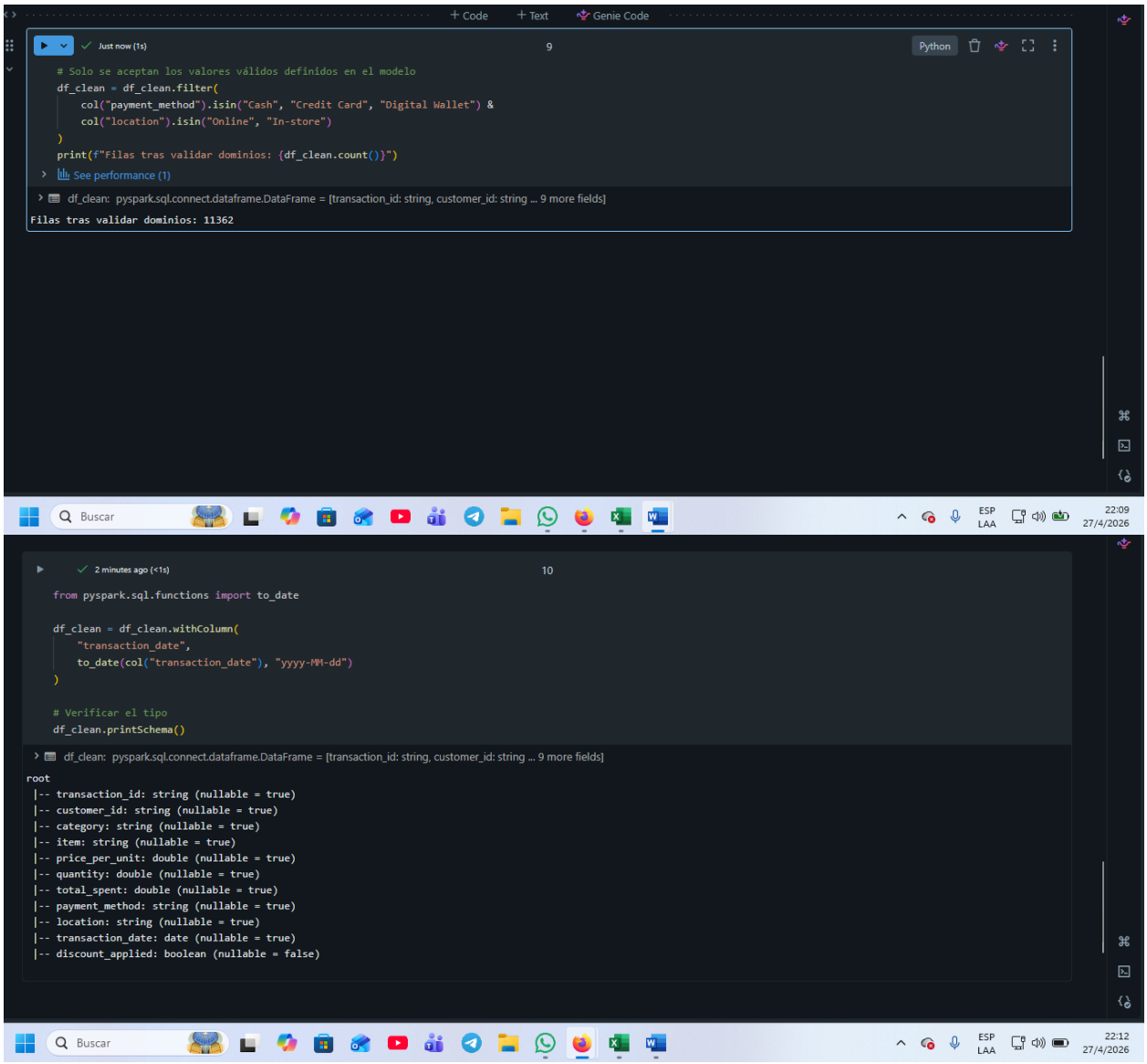

1 minute ago (1s)
7
Python
# Total Spent siempre se recalcula como price x quantity
df_clean = df_clean.withColumn(
"total_spent",
round(col("price_per_unit") * col("quantity"), 2)
)
# Mostrar 5 filas para verificar
df_clean.select("transaction_id", "price_per_unit", "quantity", "total_spent").show(5)
See performance (1)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
+--------------+--------------+--------+-----------+
|transaction_id|price_per_unit|quantity|total_spent|
+--------------+--------------+--------+-----------+
TXN_6867343	18.5	10.0	185.0
TXN_3731986	29.0	9.0	261.0
TXN_9303719	21.5	2.0	43.0
TXN_9458126	27.5	9.0	247.5
TXN_4575373	12.5	7.0	87.5
+--------------+--------------+--------+-----------+
only showing top 5 rows
Buscar
ESP
LAA
21:56
27/4/2026


Just now (1s)
5
Python
from pyspark.sql.functions import col, when, round
# Completar price_per_unit si falta
df_clean = df_clean.withColumn(
"price_per_unit",
when(
col("price_per_unit").isNull(),
round(col("total_spent") / col("quantity"), 2)
).otherwise(col("price_per_unit"))
)
# Completar quantity si falta
df_clean = df_clean.withColumn(
"quantity",
when(
col("quantity").isNull(),
round(col("total_spent") / col("price_per_unit")).cast("int")
).otherwise(col("quantity"))
)
# Verificar cuántos nulos quedan
nulos_price = df_clean.filter(col("price_per_unit").isNull()).count()
nulos_qty = df_clean.filter(col("quantity").isNull()).count()
print(f"Nulos restantes en price_per_unit: {nulos_price}")
print(f"Nulos restantes en quantity: {nulos_qty}")
See performance (2)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
Nulos restantes en price_per_unit: 0
Nulos restantes en quantity: 0

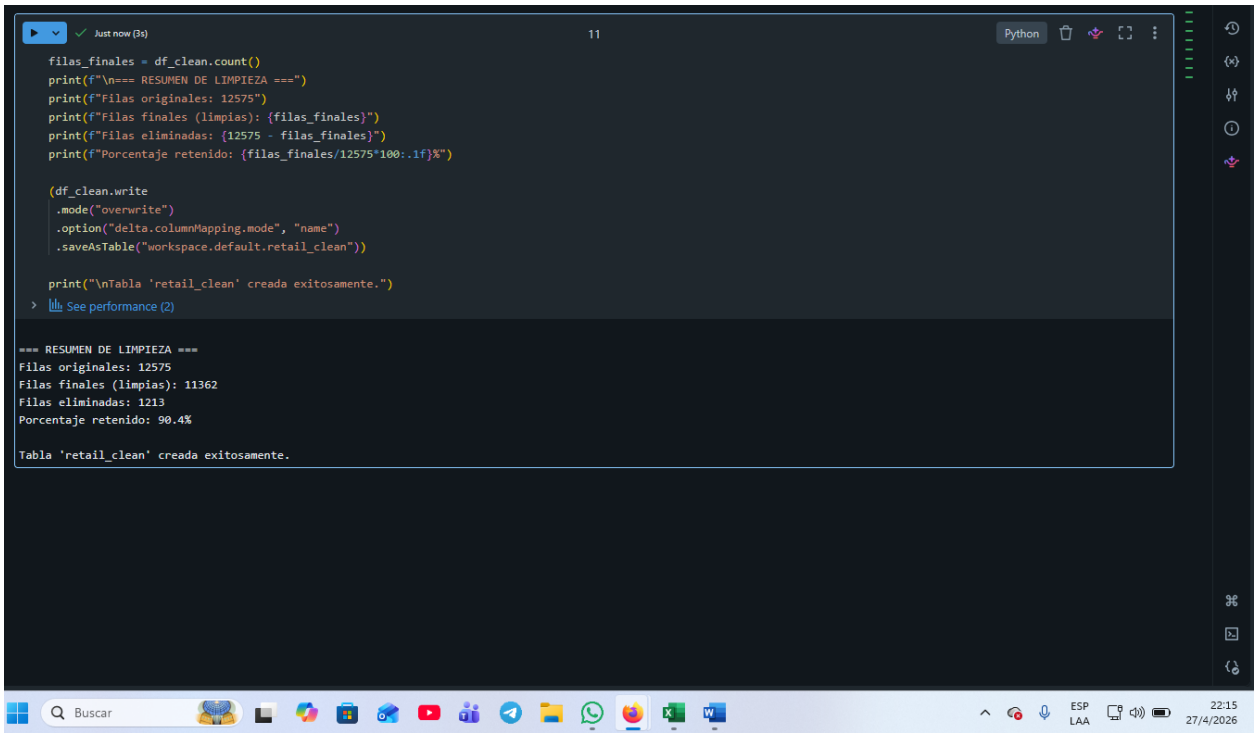

Buscar
ESP
LAA
21:49
27/4/2026
+ Code
+ Text
Genie Code
Just now (1s)
9
Python
# Solo se aceptan los valores válidos definidos en el modelo
df_clean = df_clean.filter(
col("payment_method").isin("Cash", "Credit Card", "Digital Wallet") &
col("location").isin("Online", "In-store")
)
print(f"Filas tras validar dominios: {df_clean.count()}")
See performance (1)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
Filas tras validar dominios: 11362

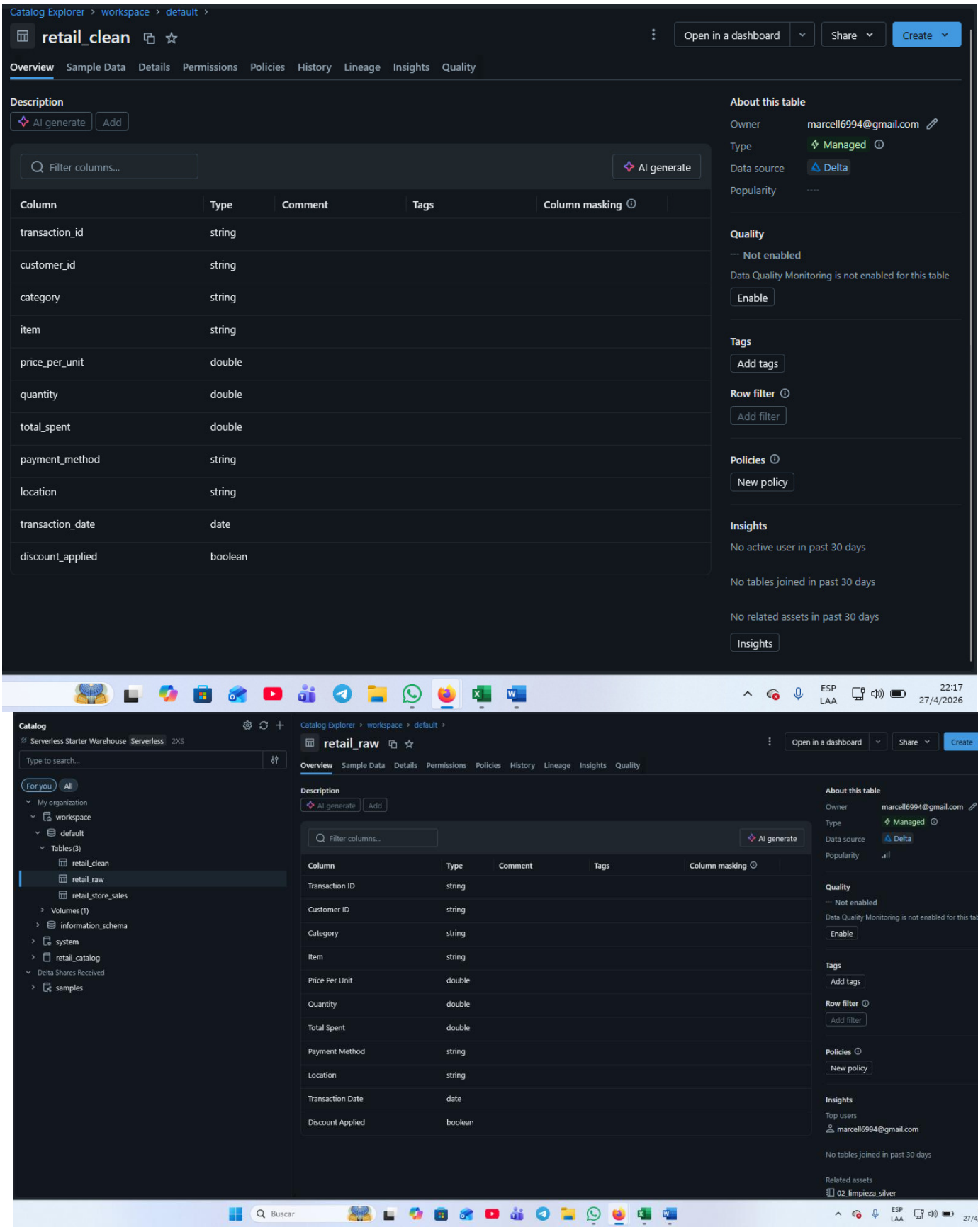

Buscar
ESP
LAA
22:09
27/4/2026
09:57 PM (1s)
8
Python
from pyspark.sql.functions import lit
# Convertir el campo a booleano y reemplazar nulos por False
df_clean = df_clean.withColumn(
"discount_applied",
when(col("discount_applied") == "True", lit(True))
.when(col("discount_applied") == "False", lit(False))
.otherwise(lit(False)) # Los nulos se asumen False
)
# Verificar la distribución
df_clean.groupBy("discount_applied").count().show()
See performance (1)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
+----------------+-----+
|discount_applied|count|
+----------------+-----+
| true| 3801|
| false| 7561|
+----------------+-----+

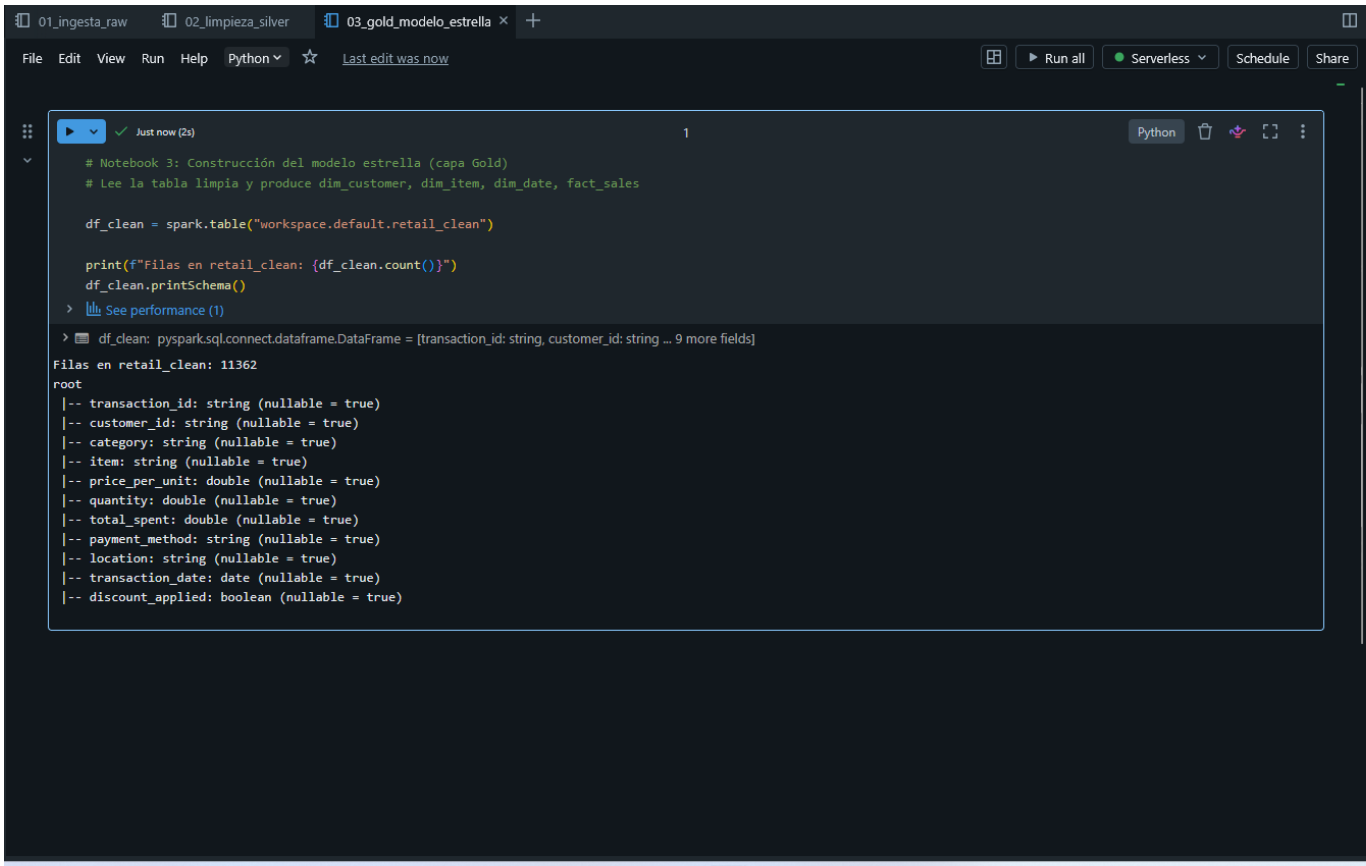
Buscar
ESP
LAA
22:03
27/4/2026

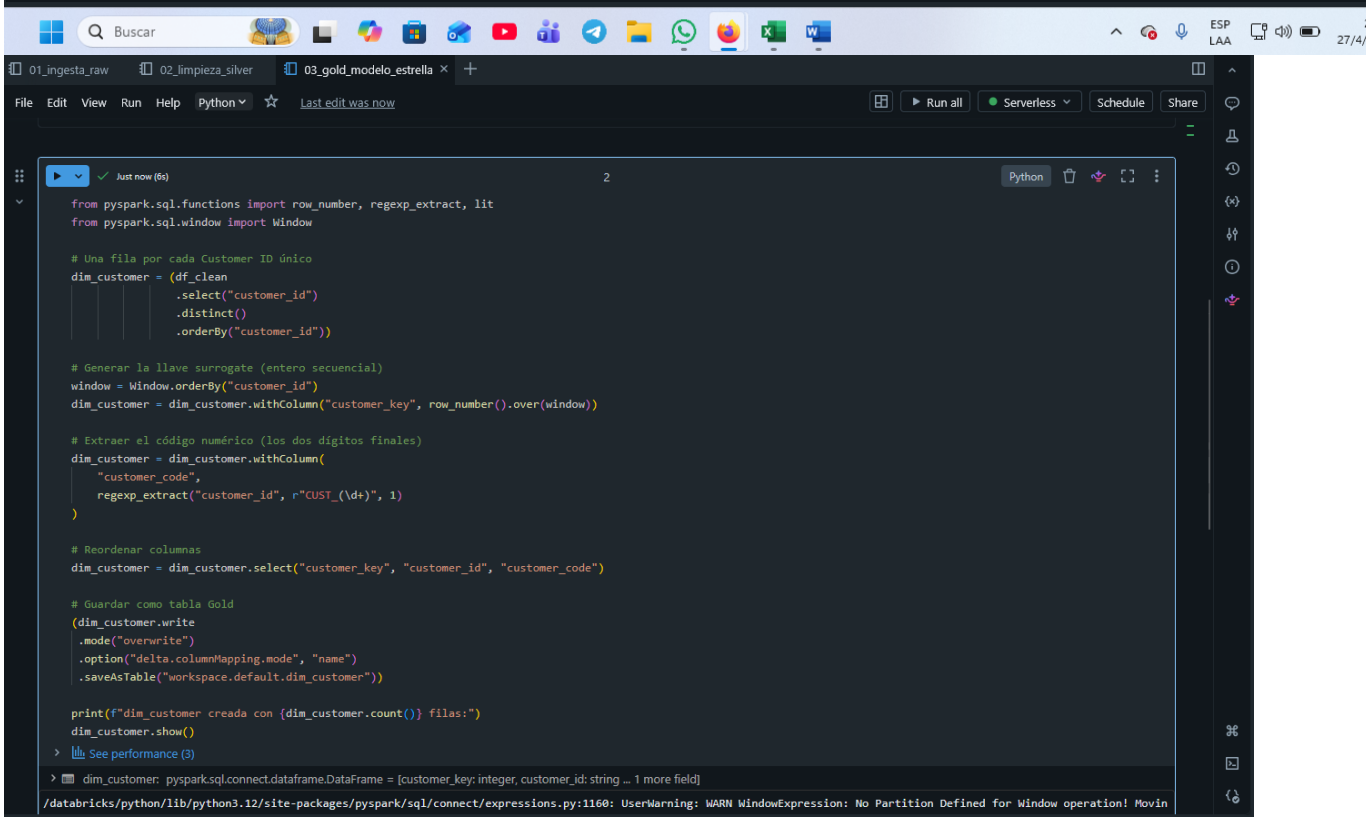

+ Code
+ Text
Genie Code
Just now (1s)
9
Python
# Solo se aceptan los valores válidos definidos en el modelo
df_clean = df_clean.filter(
col("payment_method").isin("Cash", "Credit Card", "Digital Wallet") &
col("location").isin("Online", "In-store")
)
print(f"Filas tras validar dominios: {df_clean.count()}")
See performance (1)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
Filas tras validar dominios: 11362
Buscar
ESP
LAA
22:09
27/4/2026

2 minutes ago (<1s)
10
from pyspark.sql.functions import to_date
df_clean = df_clean.withColumn(
"transaction_date",
to_date(col("transaction_date"), "yyyy-MM-dd")
)
# Verificar el tipo
df_clean.printSchema()
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
root
|-- transaction_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- category: string (nullable = true)
|-- item: string (nullable = true)
|-- price_per_unit: double (nullable = true)
|-- quantity: double (nullable = true)
|-- total_spent: double (nullable = true)
|-- payment_method: string (nullable = true)
|-- location: string (nullable = true)
|-- transaction_date: date (nullable = true)
|-- discount_applied: boolean (nullable = false)
Buscar
ESP
LAA
22:12
27/4/2026


Just now (3s)
11
Python
filas_finales = df_clean.count()
print(f"\n=== RESUMEN DE LIMPIEZA ===")
print(f"Filas originales: 12575")
print(f"Filas finales (limpias): {filas_finales}")
print(f"Filas eliminadas: {12575 - filas_finales}")
print(f"Porcentaje retenido: {filas_finales/12575*100:.1f}%")
(df_clean.write
.mode("overwrite")
.option("delta.columnMapping.mode", "name")
.saveAsTable("workspace.default.retail_clean"))
print("\nTabla 'retail_clean' creada exitosamente.")
See performance (2)
=== RESUMEN DE LIMPIEZA ===
Filas originales: 12575
Filas finales (limpias): 11362
Filas eliminadas: 1213
Porcentaje retenido: 90.4%
Tabla 'retail_clean' creada exitosamente.
Buscar
ESP
LAA
22:15
27/4/2026


Catalog Explorer › workspace › default ›
retail_clean
Open in a dashboard
Share
Create
Overview
Sample Data
Details
Permissions
Policies
History
Lineage
Insights
Quality
Description
AI generate
Add
Filter columns...
AI generate
Column
Type
Comment
Tags
Column masking
transaction_id string
customer_id string
category string
item string
price_per_unit double
quantity double
total_spent double
payment_method string
location string
transaction_date date
discount_applied boolean
About this table
Owner marcell6994@gmail.com
Type Managed
Data source Delta
Popularity
Quality
Not enabled
Data Quality Monitoring is not enabled for this table
Enable
Tags
Add tags
Row filter
Add filter
Policies
New policy
Insights
No active user in past 30 days
No tables joined in past 30 days
No related assets in past 30 days
Insights
ESP LAA
22:17
27/4/2026
Catalog
Serverless Starter Warehouse Serverless 2XS
Type to search...
For you
All
My organization
workspace
default
Tables (3)
retail_clean
retail_raw
retail_store_sales
Volumes (1)
information_schema
system
retail_catalog
Delta Shares Received
samples
Catalog Explorer › workspace › default ›
retail_raw
Open in a dashboard
Share
Create
Overview
Sample Data
Details
Permissions
Policies
History
Lineage
Insights
Quality
Description
AI generate
Add
Filter columns...
AI generate
Column
Type
Comment
Tags
Column masking
Transaction ID string
Customer ID string
Category string
Item string
Price Per Unit double
Quantity double
Total Spent double
Payment Method string
Location string
Transaction Date date
Discount Applied boolean
About this table
Owner marcell6994@gmail.com
Type Managed
Data source Delta
Popularity
Quality
Not enabled
Enable
Tags
Add tags
Row filter
Add filter
Policies
New policy
Insights
Top users
marcell6994@gmail.com
No tables joined in past 30 days
Related assets
02_limpieza_silver
Buscar
ESP LAA


01_ingesta_raw
02_limpieza_silver
03_gold_modelo_estrella
File Edit View Run Help Python
Last edit was now
Run all
Serverless
Schedule
Share
Just now (2s)
1
Python
# Notebook 3: Construcción del modelo estrella (capa Gold)
# Lee la tabla limpia y produce dim_customer, dim_item, dim_date, fact_sales
df_clean = spark.table("workspace.default.retail_clean")
print(f"Filas en retail_clean: {df_clean.count()}")
df_clean.printSchema()
See performance (1)
df_clean: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_id: string ... 9 more fields]
Filas en retail_clean: 11362
root
|-- transaction_id: string (nullable = true)
|-- customer_id: string (nullable = true)
|-- category: string (nullable = true)
|-- item: string (nullable = true)
|-- price_per_unit: double (nullable = true)
|-- quantity: double (nullable = true)
|-- total_spent: double (nullable = true)
|-- payment_method: string (nullable = true)
|-- location: string (nullable = true)
|-- transaction_date: date (nullable = true)
|-- discount_applied: boolean (nullable = true)


Buscar
ESP
LAA
01_ingesta_raw
02_limpieza_silver
03_gold_modelo_estrella
File Edit View Run Help Python
Last edit was now
Run all
Serverless
Schedule
Share
Just now (6s)
2
Python
from pyspark.sql.functions import row_number, regexp_extract, lit
from pyspark.sql.window import Window
# Una fila por cada Customer ID único
dim_customer = (df_clean
.select("customer_id")
.distinct()
.orderBy("customer_id"))
# Generar la llave surrogate (entero secuencial)
window = Window.orderBy("customer_id")
dim_customer = dim_customer.withColumn("customer_key", row_number().over(window))
# Extraer el código numérico (los dos dígitos finales)
dim_customer = dim_customer.withColumn(
"customer_code",
regexp_extract("customer_id", r"CUST_(\d+)", 1)
)
# Reordenar columnas
dim_customer = dim_customer.select("customer_key", "customer_id", "customer_code")
# Guardar como tabla Gold
(dim_customer.write
.mode("overwrite")
.option("delta.columnMapping.mode", "name")
.saveAsTable("workspace.default.dim_customer"))
print(f"dim_customer creada con {dim_customer.count()} filas:")
dim_customer.show()
See performance (3)
dim_customer: pyspark.sql.connect.dataframe.DataFrame = [customer_key: integer, customer_id: string ... 1 more field]
/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Movin

dim_customer: pyspark.sql.connect.dataframe.DataFrame = [customer_key: integer, customer_id: string ... 1 more field]

```
/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Movin
g all data to a single partition, this can cause serious performance degradation.
  warnings.warn(
dim_customer creada con 25 filas:
+-----------+-----------+-------------+
|customer_key|customer_id|customer_code|
+-----------+-----------+-------------+
|          1|    CUST_01|           01|
|          2|    CUST_02|           02|
|          3|    CUST_03|           03|
|          4|    CUST_04|           04|
|          5|    CUST_05|           05|
|          6|    CUST_06|           06|
|          7|    CUST_07|           07|
|          8|    CUST_08|           08|
|          9|    CUST_09|           09|
|         10|    CUST_10|           10|
|         11|    CUST_11|           11|
|         12|    CUST_12|           12|
|         13|    CUST_13|           13|
|         14|    CUST_14|           14|
```

1 minute ago (6s)

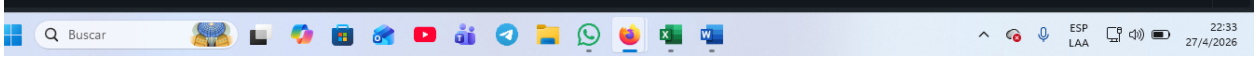
3: Cell 3

```python
from pyspark.sql.functions import col, row_number, regexp_extract
from pyspark.sql.window import Window

# Una fila por cada ítem único (incluye categoría + código de categoría)
dim_item = (df_clean
            .select("item", "category")
            .distinct()
            .orderBy("item"))

window = Window.orderBy("item")
dim_item = dim_item.withColumn("item_key", row_number().over(window))

# Extraer el sufijo del item (ej: BEV, BUT, FOOD) como category_code
dim_item = dim_item.withColumn(
    "category_code",
    regexp_extract("item", r"Item_\d+_(\w+)", 1)
)

# Renombrar columnas para alinearse con el modelo
dim_item = dim_item.withColumnRenamed("item", "item_id")
dim_item = dim_item.withColumn("item_name", col("item_id"))

# Reordenar columnas
dim_item = dim_item.select("item_key", "item_id", "item_name", "category", "category_code")

(dim_item.write
 .mode("overwrite")
 .option("delta.columnMapping.mode", "name")
 .saveAsTable("workspace.default.dim_item"))

print(f"dim_item creada con {dim_item.count()} filas:")
dim_item.show(10)
```

See performance (3)

```
/databricks/python/lib/python3.12/site-packages/pyspark/sql/connect/expressions.py:1160: UserWarning: WARN WindowExpression: No Partition Defined for Window operation! Moving
all data to a single partition, this can cause serious performance degradation.
  warnings.warn(
dim_item creada con 200 filas:
+--------+-----------+-----------+--------------------+-------------+
|item_key|    item_id|  item_name|            category|category_code|
+--------+-----------+-----------+--------------------+-------------+
|       1|Item_10_BEV|Item_10_BEV|           Beverages|          BEV|
|       2|Item_10_BUT|Item_10_BUT|            Butchers|          BUT|
|       3|Item_10_CEA|Item_10_CEA|Computers and ele...|          CEA|
|       4|Item_10_EHE|Item_10_EHE|Electric househol...|          EHE|
|       5|Item_10_FOOD|Item_10_FOOD|                Food|         FOOD|
|       6|Item_10_FUR|Item_10_FUR|           Furniture|          FUR|
|       7|Item_10_MILK|Item_10_MILK|       Milk Products|         MILK|
|       8|Item_10_PAT|Item_10_PAT|          Patisserie|          PAT|
|       9|Item_11_BEV|Item_11_BEV|           Beverages|          BEV|
|      10|Item_11_BUT|Item_11_BUT|            Butchers|          BUT|
+--------+-----------+-----------+--------------------+-------------+
only showing top 10 rows
```

4



Just now (5s) 4 Python

```python
from pyspark.sql.functions import (
    explode, sequence, to_date as f_to_date, year, quarter, month,
    dayofmonth, dayofweek, date_format, expr
)

# Generar todas las fechas entre 2022-01-01 y 2025-12-31
df_dates = spark.sql("""
    SELECT explode(sequence(to_date('2022-01-01'), to_date('2025-12-31'), interval 1 day)) AS full_date
""")

# Agregar las columnas derivadas de la fecha
dim_date = (df_dates
    .withColumn("date_key", date_format("full_date", "yyyyMMdd").cast("int"))
    .withColumn("year", year("full_date"))
    .withColumn("quarter", quarter("full_date"))
    .withColumn("month", month("full_date"))
    .withColumn("month_name", date_format("full_date", "MMMM"))
    .withColumn("day", dayofmonth("full_date"))
    .withColumn("day_of_week", dayofweek("full_date"))
    .withColumn("day_name", date_format("full_date", "EEEE"))
    .withColumn("is_weekend", expr("day_of_week IN (1, 7)"))
)

# Reordenar columnas
dim_date = dim_date.select(
    "date_key", "full_date", "year", "quarter", "month", "month_name",
    "day", "day_of_week", "day_name", "is_weekend"
)

(dim_date.write
 .mode("overwrite")
 .option("delta.columnMapping.mode", "name")
 .saveAsTable("workspace.default.dim_date"))

print(f"dim_date creada con {dim_date.count()} filas")
```

```
df_dates: pyspark.sql.connect.dataframe.DataFrame = [full_date: date]
dim_date: pyspark.sql.connect.dataframe.DataFrame = [date_key: integer, full_date: date ... 8 more fields]
dim_date creada con 1461 filas
+--------+----------+----+-------+-----+----------+---+-----------+---------+----------+
|date_key| full_date|year|quarter|month|month_name|day|day_of_week| day_name|is_weekend|
+--------+----------+----+-------+-----+----------+---+-----------+---------+----------+
|20220101|2022-01-01|2022|      1|    1|   January|  1|          7| Saturday|      true|
|20220102|2022-01-02|2022|      1|    1|   January|  2|          1|   Sunday|      true|
|20220103|2022-01-03|2022|      1|    1|   January|  3|          2|   Monday|     false|
|20220104|2022-01-04|2022|      1|    1|   January|  4|          3|  Tuesday|     false|
|20220105|2022-01-05|2022|      1|    1|   January|  5|          4|Wednesday|     false|
+--------+----------+----+-------+-----+----------+---+-----------+---------+----------+
only showing top 5 rows
```

5 Python

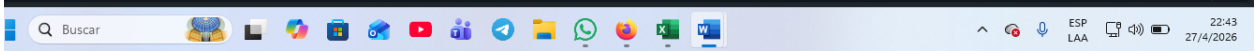

```python
# Cargar las dimensiones recién creadas
dim_customer_df = spark.table("workspace.default.dim_customer")
dim_item_df = spark.table("workspace.default.dim_item")
dim_date_df = spark.table("workspace.default.dim_date")

# Construir la tabla de hechos con los joins
fact_sales = (df_clean
    # Join con dim_customer
    .join(dim_customer_df.select("customer_key", "customer_id"), on="customer_id", how="inner")
    # Join con dim_item (la columna se llama "item" en clean, "item_id" en dim)
    .join(dim_item_df.select("item_key", "item_id"),
          df_clean["item"] == dim_item_df["item_id"], "inner")
    # Join con dim_date
    .join(dim_date_df.select("date_key", "full_date"),
          df_clean["transaction_date"] == dim_date_df["full_date"], "inner")
    # Castear quantity a INT (estaba como double)
    .withColumn("quantity", col("quantity").cast("int"))
    # Seleccionar solo las columnas finales según el modelo
    .select(
        "transaction_id",
        "customer_key",
        "item_key",
        "date_key",
        "quantity",
        "price_per_unit",
        "total_spent",
        "payment_method",
        "location",
        "discount_applied"
    )
)
```

```
    (fact_sales.write
     .mode("overwrite")
     .option("delta.columnMapping.mode", "name")
     .saveAsTable("workspace.default.fact_sales"))

    print(f"fact_sales creada con {fact_sales.count()} filas")
    fact_sales.show(5)
```

See performance (3)

dim_customer_df: pyspark.sql.connect.dataframe.DataFrame = [customer_key: integer, customer_id: string ... 1 more field]
dim_item_df: pyspark.sql.connect.dataframe.DataFrame = [item_key: integer, item_id: string ... 3 more fields]
dim_date_df: pyspark.sql.connect.dataframe.DataFrame = [date_key: integer, full_date: date ... 8 more fields]
fact_sales: pyspark.sql.connect.dataframe.DataFrame = [transaction_id: string, customer_key: integer ... 8 more fields]

```
fact_sales creada con 11362 filas
+--------------+------------+--------+--------+--------+--------------+-----------+--------------+--------+----------------+
|transaction_id|customer_key|item_key|date_key|quantity|price_per_unit|total_spent|payment_method|location|discount_applied|
+--------------+------------+--------+--------+--------+--------------+-----------+--------------+--------+----------------+
|   TXN_6867343|           9|       8|20240408|      10|          18.5|      185.0|Digital Wallet|  Online|            true|
|   TXN_3731986|          22|      63|20230723|       9|          29.0|      261.0|Digital Wallet|  Online|            true|
|   TXN_9303719|           2|      18|20221005|       2|          21.5|       43.0|   Credit Card|  Online|           false|
|   TXN_9458126|           6|      49|20220507|       9|          27.5|      247.5|   Credit Card|  Online|           false|
|   TXN_4575373|           5|     173|20221002|       7|          12.5|       87.5|Digital Wallet|  Online|           false|
+--------------+------------+--------+--------+--------+--------------+-----------+--------------+--------+----------------+
only showing top 5 rows
```



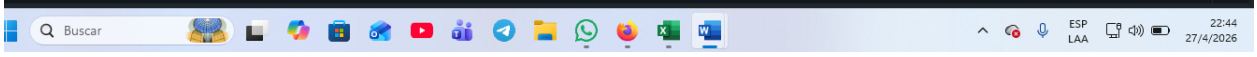
Se realiza la verificación rápida de las dimensiones:

Just now (3s) 6 Python

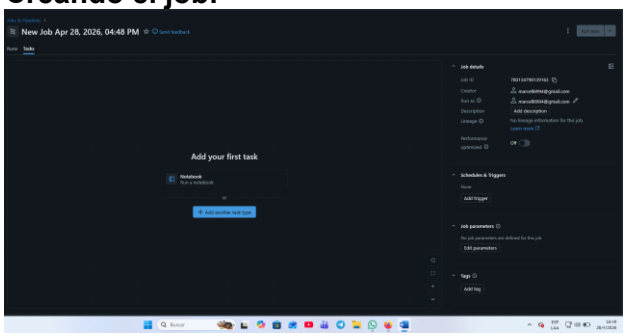
```
    # Verificar integridad del modelo estrella
    print("=== RESUMEN DEL MODELO ESTRELLA ===\n")
    print(f"fact_sales:   {spark.table('workspace.default.fact_sales').count()} filas")
    print(f"dim_customer: {spark.table('workspace.default.dim_customer').count()} filas")
    print(f"dim_item:     {spark.table('workspace.default.dim_item').count()} filas")
    print(f"dim_date:     {spark.table('workspace.default.dim_date').count()} filas")

    # Ejemplo de query OLAP: ventas totales por categoría
    print("\n=== TOP 5 CATEGORÍAS POR INGRESOS ===")
    spark.sql("""
        SELECT i.category, ROUND(SUM(f.total_spent), 2) AS total_ingresos
        FROM workspace.default.fact_sales f
        JOIN workspace.default.dim_item i ON f.item_key = i.item_key
        GROUP BY i.category
        ORDER BY total_ingresos DESC
        LIMIT 5
    """).show()
```

See performance (5)

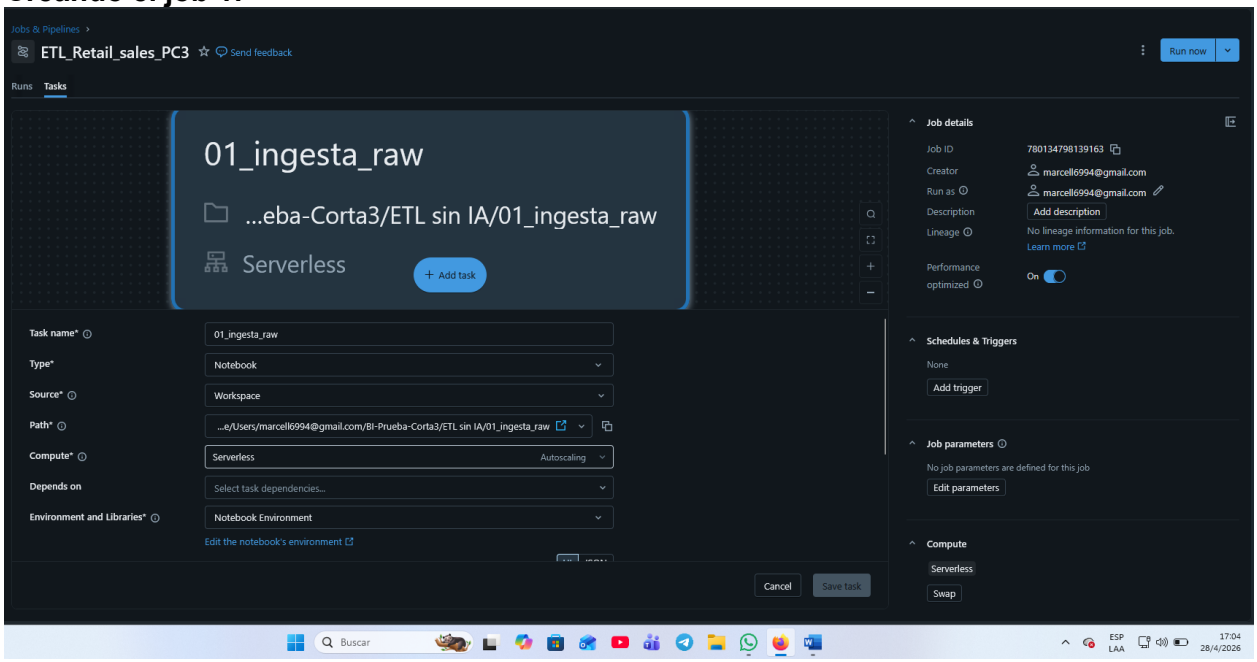
```
=== RESUMEN DEL MODELO ESTRELLA ===

fact_sales:   11362 filas
dim_customer: 25 filas
dim_item:     200 filas
dim_date:     1461 filas

=== TOP 5 CATEGORÍAS POR INGRESOS ===
+--------------------+--------------+
|            category|total_ingresos|
+--------------------+--------------+
|            Butchers|      197426.0|
|Electric househol...|      192441.5|
|           Beverages|      187978.5|
|           Furniture|      186527.0|
|                Food|      184645.0|
+--------------------+--------------+
```



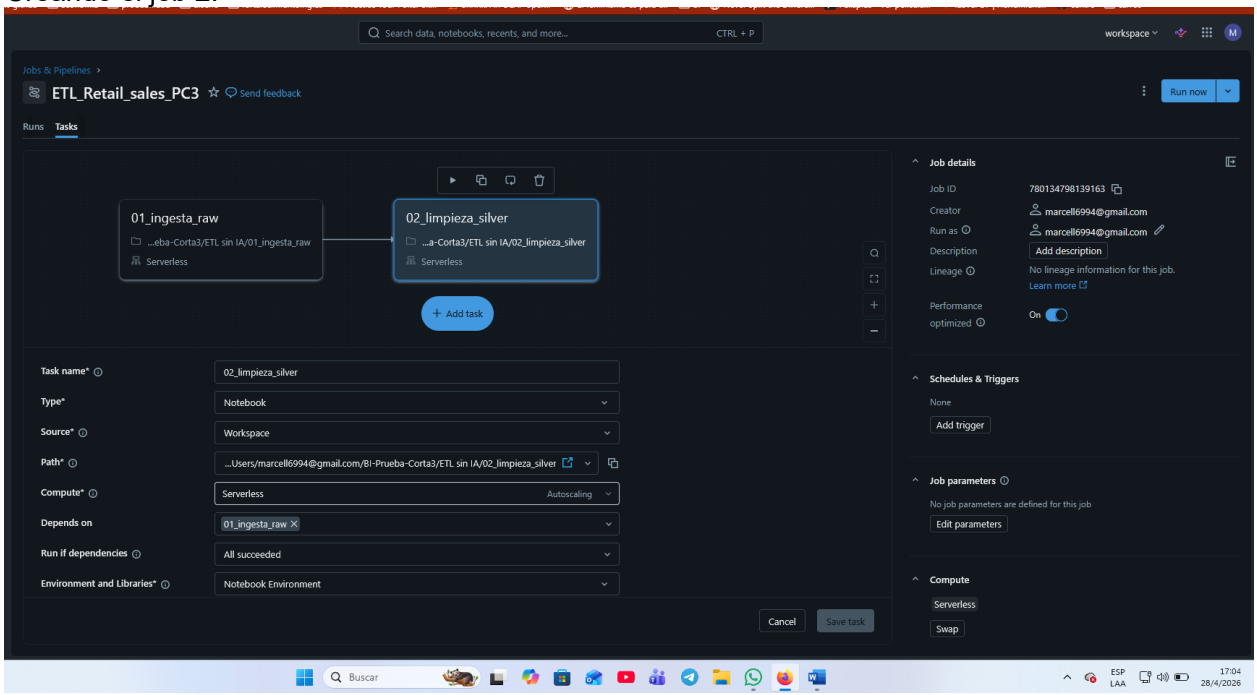
Creando el job:



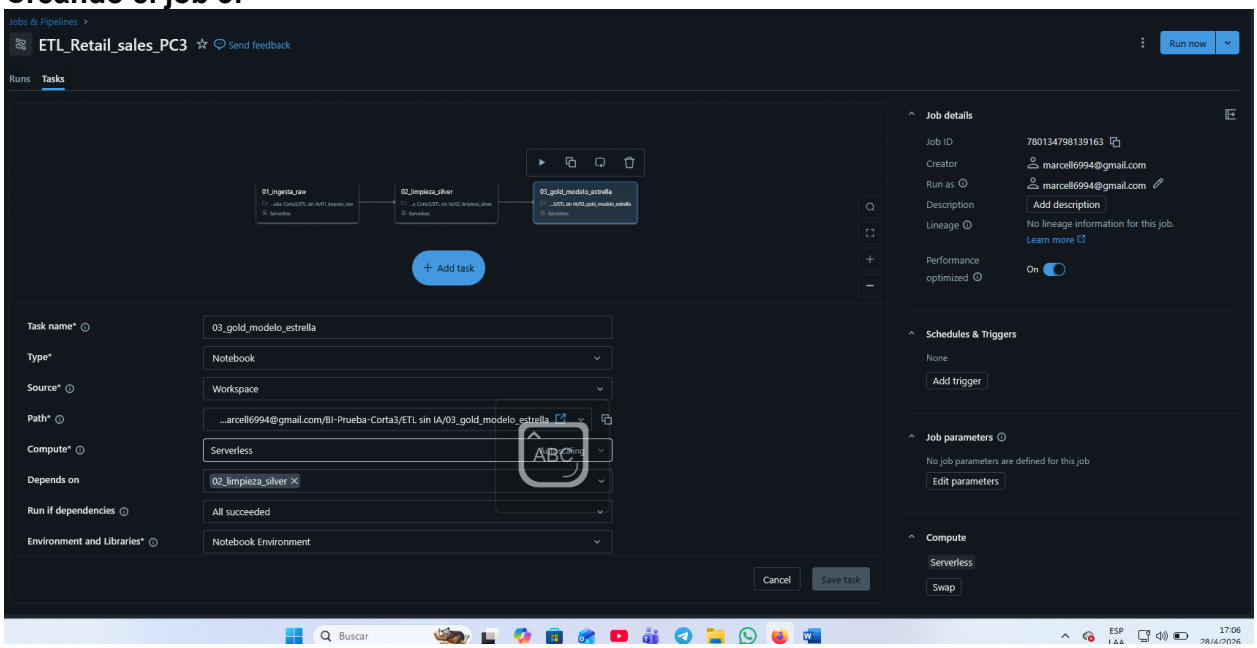
Creando el job 1:



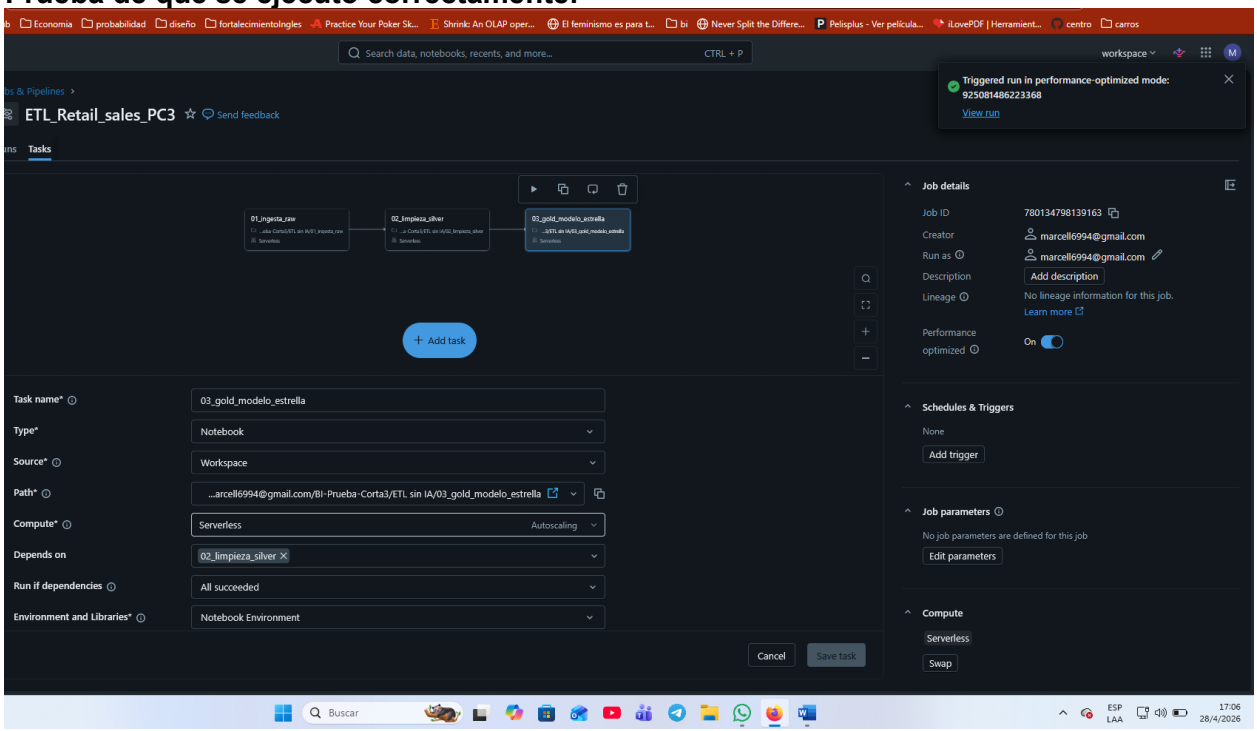
Creando el job 2:



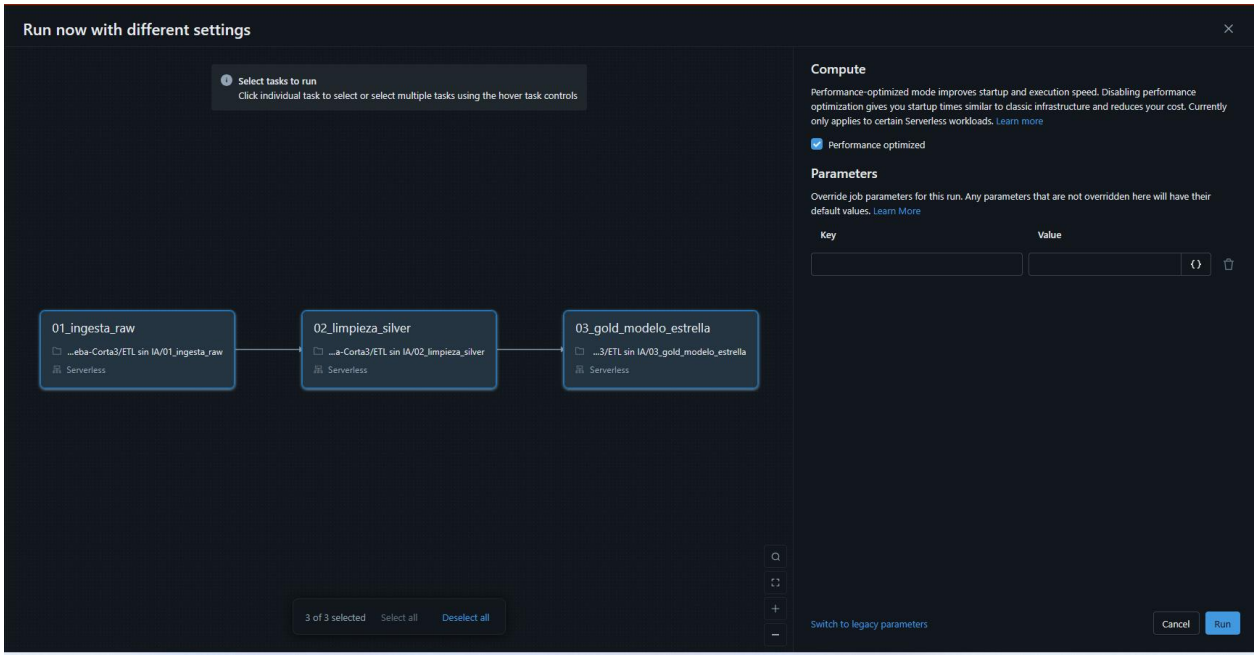
Creando el job 3:



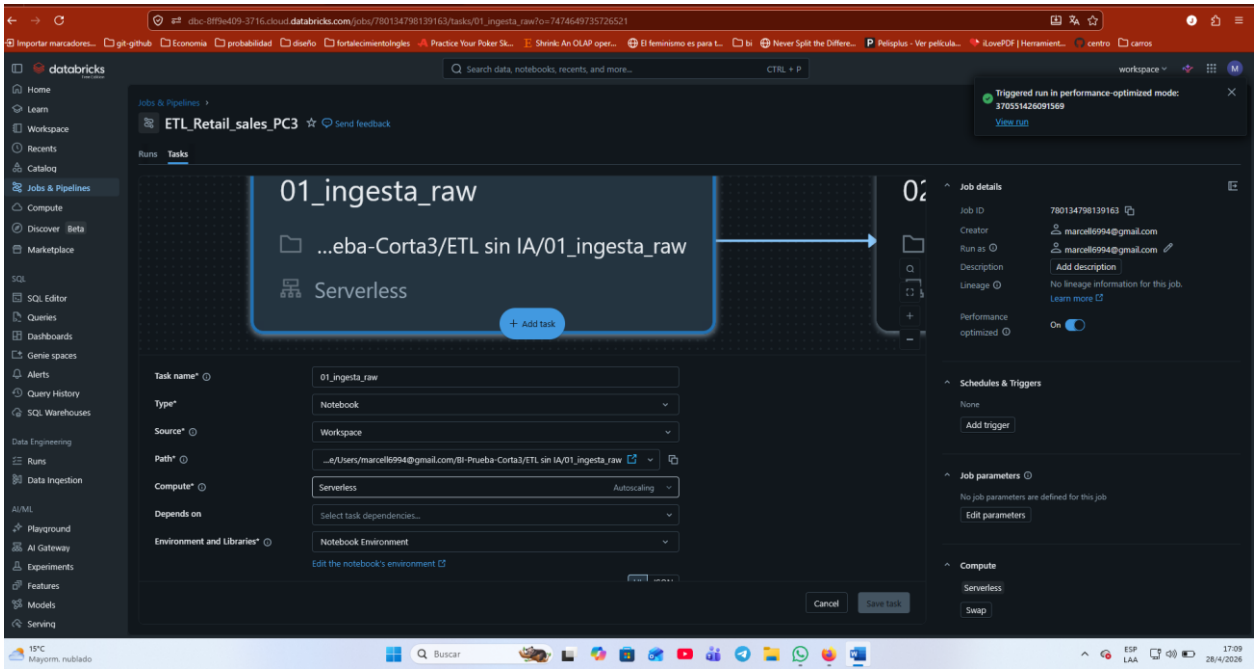
Prueba de que se ejecutó correctamente:



Antes de correr el ETL:

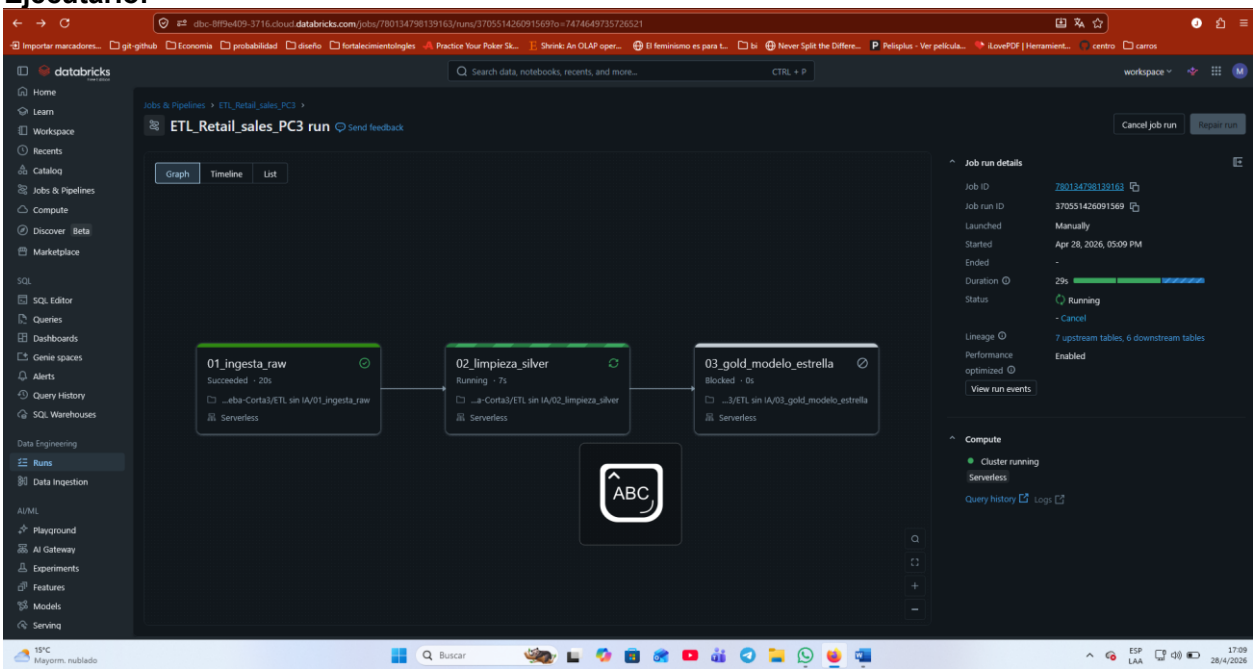


Corriendo el ETL

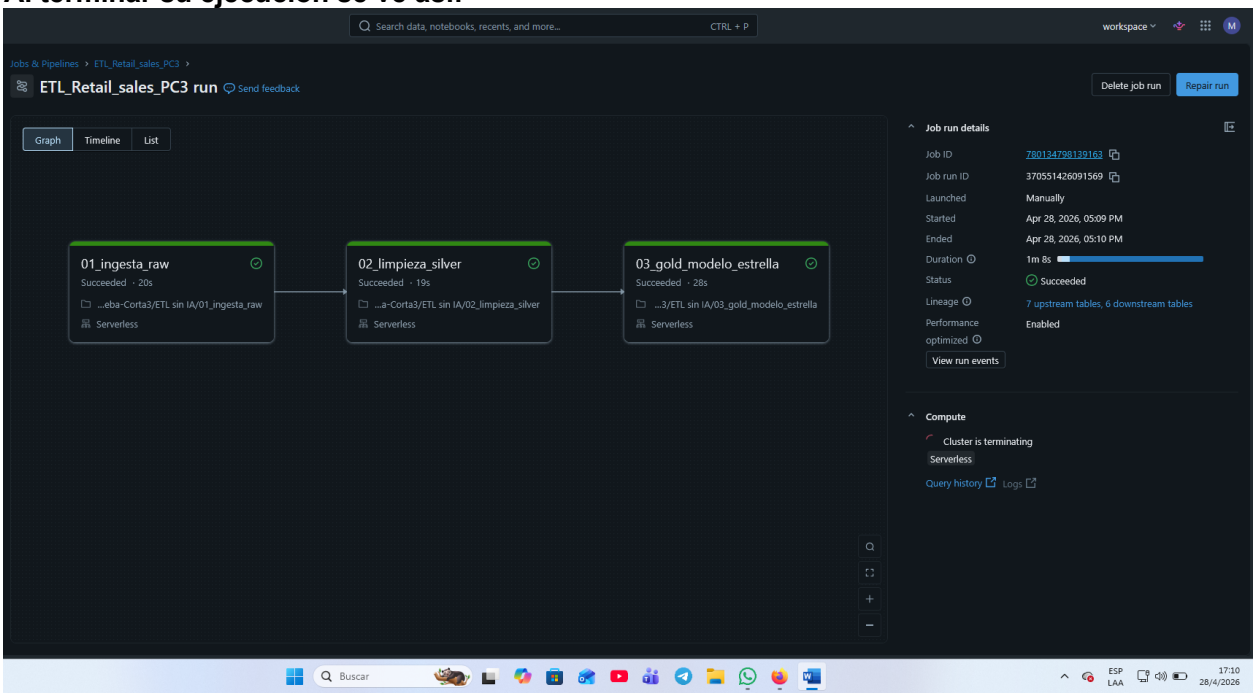


AI

Ejecutarlo:



Al terminar su ejecución se ve así:



Prueba de ejecución de cada línea: